

Sim. Pelas pesquisas dessa última semana, há um conjunto de conceitos que vale a pena você dominar porque eles aparecem repetidamente em praticamente todos os sistemas de agentes autoaperfeiçoáveis.

Uma forma boa de pensar é separar tudo em **6 camadas**: modelo, execução, conhecimento, ferramentas, avaliação e evolução.

1. Modelo / LLM

É o “cérebro bruto”.

Exemplos:

GPT-5.6 Sol

Claude

Qwen

DeepSeek

O modelo recebe contexto e produz decisões, texto, chamadas de ferramentas etc.

Quando falamos em **alterar os pesos**, estamos mudando o próprio modelo neural.

```
LLM
↓
pesos
↓
comportamento aprendido internamente
```

Fine-tuning, RL, distillation e continual learning mexem nessa camada.

2. Harness

Esse é um dos conceitos mais importantes.

O **harness** é todo o sistema ao redor do LLM que o transforma em agente.

Ele pode incluir:

Parte	Função
Prompt de sistema	regras gerais
memória	guardar conhecimento
tools	executar ações

Parte	Função
MCP	expor ferramentas
workflow	organizar etapas
retries	tentar novamente
evaluator	avaliar resultado
subagentes	dividir trabalho
logs/traces	registrar execução
segurança	limitar ações
routing	decidir modelo/tool/agente

Então:

LLM = cérebro

Harness = organismo/sistema que permite ao cérebro trabalhar

É exatamente por isso que vários papers conseguem melhorar muito o agente **sem mudar o modelo**.

3. Scaffold

É uma parte do harness.

O **scaffold** é a estrutura lógica de execução.

Por exemplo:

```
receber tarefa
→ analisar
→ planejar
→ procurar contexto
→ escolher ferramenta
→ executar
→ verificar
→ corrigir se necessário
→ responder
```

Outro scaffold poderia ser:

```
planner
↓
researcher
↓
coder
↓
tester
↓
reviewer
```

O scaffold responde:

“Como o agente trabalha?”

Enquanto o harness responde:

“Qual é toda a infraestrutura usada pelo agente?”

4. Skill

Esse é provavelmente o conceito que mais vai aparecer nos próximos anos.

Uma **skill** é um procedimento reutilizável aprendido ou criado para resolver uma classe de tarefas.

Exemplo:

```
Skill: diagnosticar erro Spring Boot
```

1. Ler stack trace
2. Localizar primeira causa
3. Verificar bean/configuração
4. Rodar testes relevantes
5. Corrigir
6. Rodar novamente

Pode ser apenas texto:

```
SKILL.md
```

ou conter código:

```
skill/
├─ SKILL.md
```

A diferença entre **memória** e **skill** é importante.

Memória:

“Na última vez o problema era timeout.”

Skill:

“Quando houver timeout, siga este procedimento.”

Ou seja:

memória = conhecimento

skill = conhecimento operacionalizado

5. Playbook

Um **playbook** é parecido com uma skill, mas normalmente é mais amplo.

Uma skill costuma resolver **uma capacidade específica**.

Um playbook descreve **como lidar com uma classe inteira de situações**.

Por exemplo:

Skill

```
validar token OAuth
```

Playbook

```
Problemas de autenticação
```

1. identificar provider
2. validar token
3. verificar expiração
4. verificar scopes
5. renovar token
6. testar chamada
7. registrar causa

Então podemos pensar:

Playbook

- └ Skill A
- └ Skill B
- └ Skill C

No Auto-RecSys da Meta, por exemplo, experiências repetidas viravam um **playbook operacional** para futuras execuções.

6. Routing

Routing significa decidir **para onde mandar determinada tarefa**.

Pode ser routing entre modelos:

```
tarefa simples  
→ modelo barato  
  
tarefa difícil  
→ GPT-5.6 Sol
```

Pode ser routing entre agentes:

```
erro Java  
→ agente Java  
  
problema banco  
→ agente SQL  
  
problema infra  
→ agente DevOps
```

Pode ser routing entre ferramentas:

```
preciso conhecer arquivo  
→ read_file  
  
preciso alterar  
→ write_file  
  
preciso testar  
→ run_shell
```

Um **router** é quem toma essa decisão.

E isso também pode aprender.

Por exemplo:

```
antes:
problemas Maven → agente geral

depois de 100 execuções:
problemas Maven complexos → agente Java especializado
```

Isso é **routing evolution**.

7. Tool

Uma **tool** é uma ação que o agente consegue executar.

Exemplos:

```
read_file
write_file
run_shell
search_database
send_email
get_campaign_metrics
deploy_service
```

O LLM sozinho não faz isso.

Ele pede:

```
run_shell("mvn test")
```

O sistema executa.

8. MCP

O **MCP Server** padroniza como essas ferramentas são disponibilizadas ao agente.

Você pode imaginar:

```
LLM
↓
Harness
↓
MCP Client
↓
```

MCP Server

↓

Tools

O MCP pode expor:

```
get_campaign  
update_campaign  
read_log  
search_docs  
execute_sql
```

Uma ideia importante dos papers recentes é que **não apenas as tools podem ser aprendidas; a forma como elas são usadas também pode aprender.**

Exemplo:

```
Tool:  
get_campaign_metrics()
```

continua igual.

Mas o agente aprende:

```
antes de chamar:  
1. verificar período  
2. buscar baseline  
3. buscar histórico  
4. depois consultar campanha
```

Isso é uma **skill de uso da tool**.

9. Workflow

Um **workflow** é uma sequência de etapas para atingir um objetivo.

Por exemplo:

```
buscar dados  
↓  
analisar  
↓  
gerar hipótese  
↓  
executar experimento  
↓
```

```
medir
↓
decidir
```

Uma skill pode fazer parte de um workflow.

E um workflow pode usar várias tools.

```
Workflow
├─ Skill
│   └─ Tools
├─ Skill
│   └─ Tools
└─ Skill
    └─ Tools
```

10. Memory

Memória é armazenamento persistente de informação obtida anteriormente.

Existem vários tipos.

Episodic memory

Lembra acontecimentos:

“Ontem tentei X e falhou.”

Semantic memory

Lembra fatos:

“API X retorna 429 quando ultrapassa 20 req/s.”

Procedural memory

Lembra como fazer:

“Para resolver 429, use exponential backoff.”

Essa terceira começa a ficar muito próxima de **skill**.

11. Retrieval

Quando há muita memória, surge outro problema:

Qual memória devo recuperar agora?

Isso é retrieval.

O agente pode possuir 10.000 experiências, mas recuperar apenas 5.

```
query
↓
retrieval
↓
memórias relevantes
↓
LLM
```

Se o retrieval aprende quais experiências ajudam mais, temos:

adaptive retrieval

ou

retrieval evolution.

12. RAG

RAG significa **Retrieval-Augmented Generation**.

O modelo recebe informação recuperada de uma fonte externa antes de responder.

```
pergunta
↓
buscar documentos
↓
selecionar trechos
↓
LLM
↓
resposta
```

RAG não significa necessariamente aprendizado.

Se o banco nunca muda:

RAG ≠ aprendizado

Mas se o agente começa a:

- criar novas entradas;
- corrigir entradas;
- remover entradas ruins;
- alterar a estratégia de retrieval;

então começa a haver **evolução do RAG**.

13. Trace

Esse conceito é absolutamente central.

Um **trace** é o histórico estruturado de tudo que ocorreu numa execução.

Exemplo:

```
task
↓
prompt
↓
tool A
↓
resultado
↓
LLM decision
↓
tool B
↓
erro
↓
retry
↓
sucesso
```

Um trace bom registra coisas como:

```
agent
model
prompt
skill
tool
arguments
response
latency
cost
error
score
result
```

Sem traces bons fica muito difícil o agente aprender corretamente.

14. Credit Attribution

Talvez um dos conceitos mais importantes das pesquisas recentes.

Significa descobrir:

“Quem foi responsável pelo sucesso ou pelo erro?”

Imagine:

```
Planejamento certo  
Tool certa  
Argumento errado
```

A tarefa falhou.

Um agente ruim pode concluir:

“Meu planejamento está errado.”

Um agente melhor faz **credit attribution**:

```
planning = ok  
tool selection = ok  
argument preparation = erro
```

Então ele altera apenas:

```
prepare_arguments()
```

Isso reduz regressões.

15. Evaluator

O **Evaluator** mede se o agente fez um bom trabalho.

Exemplos:

```
JUnit passou?  
HTTP = 200?  
CPL caiu?
```

```
usuário converteu?  
arquivo foi criado?  
resultado correto?
```

Pode ser:

determinístico

```
test passed = true/false
```

ou baseado em IA:

```
LLM-as-a-judge
```

Os trabalhos recentes estão mostrando que **evaluators determinísticos são muito importantes**, porque o agente pode aprender a enganar um LLM judge.

16. Verifier

Um **Verifier** é um avaliador mais objetivo.

Por exemplo:

```
Agent:  
"Corrigi o bug."  
  
Verifier:  
mvn test
```

Se os testes falham:

```
não corrigiu
```

O agente não decide sozinho se foi bem.

17. Evolver

O **Evolver** é o componente que propõe melhorias.

Ele olha para:

```
traces  
falhas
```

scores
memória

e produz algo como:

nova skill
novo prompt
novo workflow
novo routing
novo código
nova tool

Em um agente autoaperfeiçoável:

Executor
↓
Trace
↓
Evaluator
↓
Evolver
↓
Candidate

18. Candidate

Uma **candidate** é uma versão nova ainda não aprovada.

Exemplo:

skill-v17

ou:

harness-v42

Ela ainda não é produção.

Precisa passar por testes.

19. Gate

O **gate** é uma regra que decide se a candidate pode avançar.

Podem existir vários:

```
Quality Gate
Safety Gate
Regression Gate
Cost Gate
Security Gate
```

Por exemplo:

```
melhorou sucesso?
SIM

piorou custo?
NÃO

quebrou segurança?
NÃO

regrediu outras tarefas?
NÃO

→ promover
```

20. Promotion

Promotion significa tornar a candidata a versão oficial.

```
v41
↓
candidate v42
↓
tests
↓
PASS
↓
v42 vira production
```

Nos sistemas que estamos acompanhando, essa é uma das partes mais críticas.

21. Rollback

Se a versão nova piorou alguma coisa:

```
v42
↓
problema
↓
rollback
↓
v41
```

Por isso versionamento é tão importante.

Um agente autoaperfeiçoável sem rollback é perigoso operacionalmente.

22. Held-out

Um **held-out set** é um conjunto de testes que o Evolver **não conhece**.

Isso é fundamental.

Se ele aprende usando:

```
100 tarefas
```

você mede em:

```
outras 100 tarefas
```

que ele nunca viu.

Assim você descobre se houve **generalização** ou apenas memorização.

23. Canary

Um **canary** é um teste deliberadamente colocado para detectar comportamento indevido.

Por exemplo:

```
candidate parece excelente
```

mas existe uma tarefa que só passaria se o agente acessasse informação proibida.

Se ele passa:

suspeita de cheating

É como uma armadilha de segurança.

24. Replay

Replay significa executar novamente uma experiência passada com uma versão nova.

```
trace antigo
↓
harness novo
↓
comparar
```

Isso permite saber:

“Essa alteração teria melhorado realmente os casos que falharam?”

25. Procedural Graph

Um **procedural graph** representa como procedimentos se relacionam.

Algo como:

```
A
↓
B
↓
C
```

Mas com condições:

```
A --se erro 401--> RefreshToken
A --se 429--> Backoff
A --se 200--> ProcessResponse
```

O agente pode aprender novas arestas.

É quase um **mapa operacional aprendido**.

26. Routing Graph

Parecido, mas usado para escolha de agentes/tools/models.

```
task
```

```
|— Java → AgentJava
```

```
|— SQL → AgentDB
```

```
|— Infra → AgentOps
```

Com experiência, os pesos/arestas desse grafo podem mudar.

27. Policy

Uma **policy** é simplesmente:

“Dado este estado, o que eu faço?”

Formalmente:

```
state → action
```

Em agentes:

```
erro 429
```

```
→ retry com backoff
```

ou:

```
problema complexo
```

```
→ chamar modelo mais forte
```

Skills, prompts e routing são maneiras de representar policies.

28. Self-play

No **self-play**, uma IA produz desafios para outra IA — ou para outra versão dela mesma.

Exemplo:

```
Red Agent
```

```
↓
```

```
encontra ataque
```

```
Blue Agent
```

```
↓
```

cria defesa

Red Agent

↓

tenta quebrar defesa nova

Isso pode gerar experiências indefinidamente.

29. Continual Learning

Significa aprender continuamente conforme novas experiências chegam.

segunda

→ aprende A

terça

→ aprende B

quarta

→ aprende C

sem esquecer A e B.

O grande problema é **catastrophic forgetting**:

aprende C

↓

esquece A

30. Online Learning

Parecido com continual learning, mas enfatiza aprender **durante a operação**.

produção

↓

experiência

↓

atualização

↓

produção seguinte

LinkedIn, Tencent e outros casos que estamos acompanhando caminham nessa direção.

31. Self-Improving Agent

É um agente que usa resultados das próprias execuções para mudar persistentemente alguma parte de si.

Pode mudar:

```
memory
skills
prompts
workflow
routing
tools
harness
pesos
```

Não precisa necessariamente mexer nos pesos.

32. Recursive Self-Improvement — RSI

Esse é um nível muito mais forte.

Não é simplesmente:

```
AI melhora uma skill.
```

Seria:

```
AI1
↓
melhora o mecanismo que produz AI
↓
AI2 melhor
↓
AI2 usa sua capacidade superior
para melhorar o mecanismo novamente
↓
AI3 ainda melhor
↓
...
```

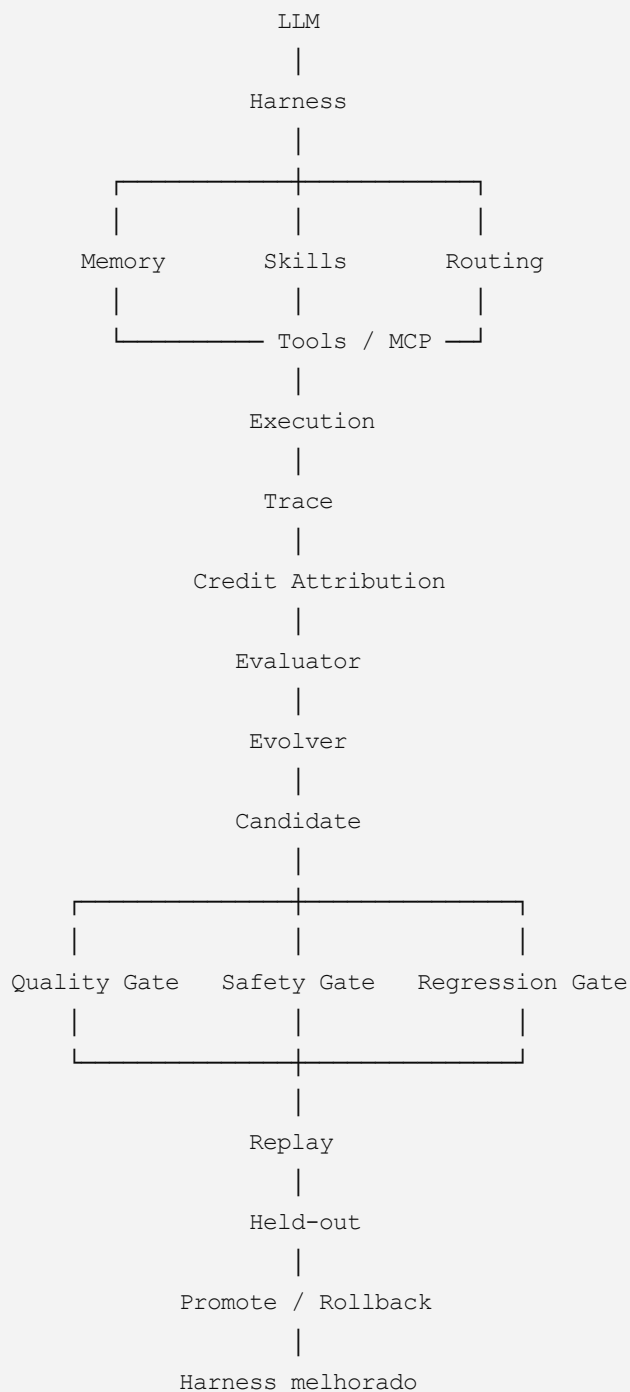
O elemento fundamental é:

a melhoria aumenta a capacidade de produzir a próxima melhoria.

Ainda não vimos uma demonstração convincente de RSI aberto e sustentado.

A arquitetura completa

Juntando praticamente tudo que apareceu no radar:



Esse desenho resume **boa parte do que estamos vendo surgir na pesquisa de agentes em 2026**.

E, pensando em algo que você possa efetivamente construir, eu priorizaria conhecer profundamente estes **10 conceitos**:

Harness → Trace → Skill → MCP Tool → Memory → Routing → Credit Attribution → Evaluator → Evolver → Promotion/Rollback.

Com esses dez você já consegue compreender a maior parte dos papers e sistemas que estamos acompanhando.